Persistent Online False-Discovery Control for a Guarded Self-Improvement Loop: A Process-Restart Bug, Its Fix, and a Pilot Case Study

Abdelkabir Ouadoukou*

November 2026

si-loop

Abstract

Study 3 [7] ran a live language-model proposer behind SI-Loop's sealed judge, with an online false-discovery monitor (e-LOND, 12) attached in *monitor* mode: it logged alarms but never blocked an accept. Study 4 asks the next question on that kernel's own roadmap: can the loop also improve the code that proposes and judges its own changes (arm A1), under the same monitor now running in *enforced* mode. Building the arm-A1 runner surfaced three bugs, none in the kernel itself. The one this paper is about: `kernel/orchestrator.py` constructs a fresh online false-discovery budget, ELond(t=0, d=0), every time a new `Orchestrator` object is built, which happens every time the command-line runner is launched as a new process. Within one continuous run the budget tightens correctly; across the separate process launches a long-lived pilot is actually made of, it silently resets, handing back a generous early-test allowance each time and undermining the anytime-valid guarantee the monitor exists to provide. We fix this with `study4/fdr_replay.py`: a function that replays every past correctness-path test already recorded in the run's own JSON ledger, using the hit/no-hit outcome the monitor itself recorded, and injects the reconstructed state through `TwoStageJudge`'s existing optional constructor parameter, with no edit to `kernel/`. We verify the fix two ways: a unit test that builds two independent `Orchestrator` instances sharing one ledger and checks the second continues the first's budget exactly; and a live run, where we computed the next cycle's required significance level from the ledger *before* running it (0.0012158542...) and the live run matched it exactly. A 13-cycle live pilot against `openai/gpt-oss-120b` (arm A1) then shows both the bug and the fix side by side in the same ledger: one process boundary where the level incorrectly repeats, and a later one where it correctly continues. The pilot accepted nothing. Seven candidates reached the correctness path and were blocked by the enforced monitor, including one with $p_{\text{better}} = 0.9999$, because zero prior discoveries had earned the sequence any room; the required level fell from 0.0304 to 0.00062 over those seven tests. We report this as the intended, anytime-valid behaviour of a cold-started budget, not a malfunction, and we make no claim about recursive self-improvement capability: this is an infrastructure correctness result and a transparent, mid-session pilot, in the spirit of Study 3's own disclosed judge defects.

1 Introduction

A guarded self-improvement loop that runs for a long time, or across many separate invocations, tests many candidate patches sequentially. Each test is a statistical decision: does this patch's

*Independent researcher. Contact: abdelkabir.ouadoukou@gmail.com. Substantial AI assistance was used; see the disclosure at the end.

measured pass-rate gain reflect a real improvement, or noise. If the number of tests is fixed in advance, classical multiple-comparisons correction applies. An open-ended loop has no such fixed number: the stream of tests is generated adaptively, by a proposer whose next move depends on the outcome of the last one, and it can run indefinitely. This is exactly the setting *online* false-discovery control is built for [8, 12]: control the long-run false-discovery rate over an unbounded, adaptively generated sequence of tests, validly at every stopping time, not just at a horizon fixed beforehand.

Studies 2 and 3 [6, 7] built this monitor into SI-Loop's two-stage judge but ran it in *monitor* mode: every correctness-path test is still scored and logged, but a failing e-value never blocks the accept. The monitor was observational, not a gate. Study 4 asks whether the loop can go further and improve the code that proposes and judges its own changes (arm A1, editing a small policy file under `agent/meta/`), and its own design decisions include switching this specific monitor to *enforced* mode: closing exactly the gap Study 3 left open.

The problem this paper reports. Enforcing a monitor is only as good as the state it enforces against. We found, while wiring the arm-A1 runner, that the kernel's online false-discovery budget is not a property of the *study*; it is a property of the *process*. `Orchestrator.__init__` (`kernel/orchestrator.py`) always constructs `self.elond = ELond(cfg.fdr_alpha)`, a fresh object with its two counters, t (tests so far) and d (discoveries so far), both zero. Nothing anywhere persists or restores them. A long pilot is not one process; it is several, launched minutes or hours apart as rate limits, pauses, and ordinary command-line use dictate. Each launch silently re-granted the generous initial allowance an online-FDR schedule reserves for a sequence's first few tests, exactly the thing the monitor is supposed to prevent happening more than once.

Contributions.

- A precise statement and live demonstration of the process-restart gap in `kernel/stats.py`'s `ELond`, a pre-existing property of the kernel shared by Studies 2 and 3, not introduced by this work (Section 2).
- A fix, `study4/fdr_replay.py`, that reconstructs the exact (t, d) state from the ledger's own history and injects it through an existing, unused extension point in `kernel/pipeline.py`'s `TwoStageJudge`, with no edit to `kernel/` (Section 3).
- Two independent verifications of the fix: a controlled unit test, and a live run whose first required significance level was predicted from the ledger before the run and matched exactly (Section 3).
- A 13-cycle live pilot (arm A1, `openai/gpt-oss-120b`) that captures the bug and the fix in the same ledger, and documents what an enforced, cold-started online-FDR budget does to a real, adaptively generated test sequence: nothing was accepted, and the required bar tightened by roughly a factor of fifty over seven tests (Section 4).

This is an infrastructure-correctness paper with one honest case study attached, not a result about language-model self-improvement. No patch was accepted in the pilot, the pilot's own code changed mid-session as these bugs were found, and we do not pool it with any pre-registered claim; that is the same stance Study 3 took toward its own pilot run and toward the two judge defects (D1, D2) it found and disclosed.

2 Design evolution: Studies 1 through 3

Study 1 [5]. A toy closed loop (hypernetwork, curriculum, guarded judge, competitive workspace) on synthetic few-shot regression. Its main lesson for this line of work was methodological: an over-cautious judge that rejects most rounds can look simply unhelpful in aggregate statistics, and a rising curve on its own does not establish that improving the *improver* was what helped. Study 1 has no online false-discovery control; its gate is a single fixed-sample test per round.

Study 2 [6]. Introduced the kernel this work still runs on: zero-trust git worktrees, a seven-layer patch defence, a two-stage paired non-inferiority gate (an anytime-valid screen on batch A, then a strict confirm on an independent batch B), a frozen cycle-0 anchor with a margin budget, a held-out problem family, a three-tier sealed verifier with honeypots, and a hash-chained ledger with circuit breakers. `kernel/stats.py`'s `ELond` (e-LOND online false-discovery control [12], valid under the arbitrary dependence an adaptive loop produces) was built in at this stage, wired into `kernel/pipeline.py`'s confirm stage, but exercised only on a synthetic world with a scripted proposer. `cfg.fdr_mode` already had three values, `off`, `monitor`, and `enforce`; Study 2's own ablation runs used `monitor`.

Study 3 [7]. Replaced the scripted proposer with a live model (`openai/gpt-oss-120b` via Groq) on arm A0 (solver patches only), in six pre-registered replicates. Every replicate produced at least one correctness-path accept; transfer to the held-out family was rare (one of six met the pre-declared criterion). Two judge defects were found and disclosed: D1, a utility-path clause that was always true because the verifier reports zero tokens for every candidate; and D2, a verifier suite that missed a tie-break bug in one task family. Relevant here: `fdr_mode` stayed at `monitor` throughout Study 3. Its own discussion names this directly: *"The monitor that could have flagged overfitting ran in monitor mode, and we have not extracted its flags for every run. It should be enforced . . . before any stronger result is made."* Study 4's design draft (`study4/DESIGN_DRAFT.md`) accordingly declares enforcement a precondition, not an option, before any improver-arm run: this paper is the first time `fdr_mode="enforce"` has been exercised against a live, adaptively generated proposal stream on this kernel.

3 The process-restart gap and its fix

3.1 How the online budget works

`ELond` (`kernel/stats.py`) implements e-LOND [12]. Test t is declared a discovery iff its e-value clears

$$\alpha_t = \alpha \cdot \gamma_t \cdot (D_{t-1} + 1), \qquad \gamma_t = \frac{6}{\pi^2 t^2} \text{ (the default, anytime-valid schedule, with } \textstyle\sum_t \gamma_t \le 1),$$

where D_{t-1} is the number of discoveries strictly before test t. Two counters are enough to reproduce the whole trajectory: t, the number of tests run, and $d = D_t$, the number of discoveries so far. `level()` returns the level the *next* test will be run at, from the current (t, d); `test(log_e)` increments t, compares the supplied e-value against that level, and increments d on a hit. `kernel/pipeline.py`'s `TwoStageJudge` calls `self.elond.test(log_e_up(b,c))` once for every candidate that reaches the correctness path on its confirm batch, and (when `fdr_mode="enforce"`) rejects outright on a miss, regardless of what the ordinary paired gate decided.

3.2 The gap

`Orchestrator.__init__` always runs `self.elond = ELond(cfg.fdr_alpha)`. No code path anywhere calls `ELond`'s own `state()`/`load()` methods, which exist and are unused. A fresh `Orchestrator`, built every time `run_study4.py -run` is launched as a new process, therefore always starts at $(t, d) = (0, 0)$, regardless of how many correctness-path tests the same output directory's ledger already records. This is a property of `kernel/orchestrator.py` and `kernel/pipeline.py` as they stood for Studies 2 and 3 as well; it was simply never exercised in `enforce` mode before, and monitor mode's own correctness does not depend on the budget being right (it never gates on it).

3.3 The fix

`study4/fdr_replay.py` provides two functions.

- `replay_elond_state(ledger_path, fdr_alpha)` returns a fresh `ELond(fdr_alpha)` advanced through every past `"end"` ledger record that carries a `detail.fdr` entry, in file order. For each such record it applies *exactly* `ELond.test`'s own update rule, `t += 1; d += int(hit)`, using the `hit` boolean the monitor itself already recorded, not a recomputed e-value. The result is, by construction, identical to the state a single continuously running process would have reached after the same sequence of tests.
- `attach_persistent_fdr_budget(orch, ledger_path)` calls the above and assigns the result to *both* `orch.elond` and `orch.judge.elond`. These are the same object at construction time (`TwoStageJudge` is built with `self.elond` as an argument), but replacing only one of the two attributes after the fact would leave the orchestrator and the judge looking at two different budgets: a mistake worth naming because it is easy to make and silent if made.

No file under `kernel/` is edited. `TwoStageJudge.__init__` (`kernel/pipeline.py`) already accepts an optional, pre-built `elond` parameter; this is a deliberate extension point, not a workaround. `study4/run_live.py` calls `attach_persistent_fdr_budget` immediately after constructing the `Orchestrator`, before any cycle runs, for both arms A0 and A1.

4 Pilot case study

A pilot was run against the live model (`openai/gpt-oss-120b` via Groq), arm A1, over four separate process launches in one session, output directory `runs/study4_live`. **This is a debugging-phase pilot, not a pre-registered replicate.** The code itself changed partway through: a local self-test environment issue was fixed by setting up the project's virtual environment, `study4/yield_gate.py`'s dedicated-synthetic-source fix was applied, and this paper's own fix was built and wired, all *during* this session, the same way Study 3's pilot exposed D1. It is reported in full, kept unpooled, the same stance `results/study3/DEVIATIONS.md` takes toward that pilot.

4.1 Overall outcome

Thirteen cycles ran; none was accepted (Table 1). Every judged cycle proposed a change to `agent/lib.py` (scope `solver`); arm A1's live model never attempted an edit to `agent/meta/policy.py` in this pilot, despite its system prompt stating that path is allowed. With only 13 cycles this is not evidence either way, only something to watch in a longer run.

Table 1: All 13 cycles of the pilot, by rejection cause.

Reject reason	Count	Cause
`selftest`	2	pre-fix environment bug (sandboxed self-test could not see a user-site `pytest`); not a judge outcome
`fdr`	7	enforced online false-discovery control (this paper's subject)
`no_gain`	2	ordinary two-stage-judge rejection, no measured pass-rate improvement
`unconfirmed`	2	screened well on batch A, did not hold up on independent batch B

4.2 The bug and the fix, side by side, in one ledger

Seven candidates reached the correctness path and were tested by `ELond`. Table 2 lists all seven in ledger order, with the process boundary each one followed. Tests 1 and 2 are separated by a process restart under the *pre-fix* code: test 2 incorrectly repeats test 1's level exactly, 0.030396…, instead of the lower level a continuing sequence would use: the empirical signature of the bug. Tests 2, 3, and 4 run inside one process (the restart gap being measured is specifically *across* launches, not within one), and correctly tighten relative to each other. Tests 4 and 5 are separated by a second process restart, this time under the *fixed* code: test 5's level was computed from the ledger *before* that run (predicted 0.0012158542… from t=4, d=0), and the live run matched it exactly.

Table 2: Every correctness-path test in the pilot, in ledger order. p_{better} and δ are the ordinary paired gate's own statistics on that candidate's confirm batch ($n = 48$); neither cleared α_t (column 'level'), so every row is a miss ('hit' = false) under the enforced monitor, regardless of how convincing the ordinary gate found it.

Test	p_{better}	δ	Level α_t	Boundary crossed before this test	Correct?
1	0.9688	+0.083	0.030396	(first test of the pilot)	–
2	0.9688	+0.083	0.030396	**process restart, pre-fix**	**no (repeats test 1)**
3	0.9922	+0.125	0.007599	(same process as test 2)	yes
4	0.9961	+0.146	0.003377	(same process as test 3)	yes
5	0.9688	+0.083	0.001216	**process restart, post-fix**	**yes (predicted in advance)**
6	0.9922	+0.125	0.000844	(same process as test 5)	yes
7	0.9999	+0.250	0.000620	(same process as test 6)	yes

Figure 3 plots the required level against test index on a log scale. Final state: $(t, d) = (7, 0)$; the next correctness-path test in this ledger needs $\alpha_8 \approx 0.000475$, an e-value above roughly 2100, to be declared a discovery.

4.3 A concrete rejected candidate

Test 4's candidate, inspected directly in `runs/study4_live/runs/proposal_history.jsonl`, implemented two functions the seed deliberately leaves as `raise NotImplementedError` placeholders, `two_sum` and `flatten`, both correctly and without hard-coding, matching their docstrings exactly. Its confirm-batch statistics, $b = 4, c = 0, n = 48$, $p_{\text{better}} = 0.9961$, would clear the ordinary gate's own correctness-path threshold ($p_{\text{improve_min}} = 0.95$) comfortably. The enforced monitor rejected it anyway, because the cumulative budget at that point required $\alpha_4 = 0.003377$ and the candidate's e-value did not reach it. The mechanism blocking this candidate was the statistics, not a flaw in the candidate.

5 Discussion and future work

What this shows. An enforced, cold-started e-LOND budget does exactly what it is specified to do: it does not treat a single convincing-looking test, however convincing, as a confirmed discovery when the sequence has not yet earned the room for one. Table 2's test 7 ($p_{\text{better}} = 0.9999$) is the clearest illustration available in this pilot. This directly closes the gap Study 3 named in its own limitations: a monitor that only logs cannot have this effect, because it never gates.

What it does not show. Thirteen cycles, zero discoveries, is not evidence that arm A1 cannot produce an accepted patch, nor that the enforced setting is miscalibrated. With $\gamma_t \propto 1/t^2$ and $d = 0$ throughout, the budget available to test t shrinks whether or not the candidates being tested are improving, and Table 2 shows they were getting *more* convincing (δ rising from 0.083 to 0.250) while the bar rose faster still. This is the known cost of anytime-valid control under a cold start: safety is bought with power, and the only thing that replenishes the budget, a factor of $(d + 1)$ on every subsequent α_t, is an actual discovery. A sequence that never gets one stays conservative by design.

A trap to name and avoid. The natural-looking response to a starved budget, lowering $p_{\text{improve_min}}$ or α until something clears the bar, is exactly the kind of post-hoc, result-driven parameter change the pre-registration discipline in this repository exists to prevent (Studies 2 and 3 record their own post-hoc deviations explicitly rather than making them silently). Any recalibration belongs in a new pre-registration, decided before seeing further results, not as a reaction to this pilot's zero.

Mitigations worth evaluating, not yet adopted. `ELond` already supports a `uniform` schedule ($\gamma_t = 1/\text{horizon}$) as an alternative to the default `inv_sq`; a declared horizon trades the indefinite-runtime guarantee for a flatter, less front-loaded budget, which may suit a pilot of known length better than the anytime-valid default suits an open-ended one. A burn-in of a fixed number of cycles excluded from enforcement (but still logged) would need its own pre-registration and its own false-discovery accounting, not an ad hoc exemption. Both are future design decisions, not changes made in this paper.

Future work. (i) A pre-registered replicate set, run only now that the three bugs in this paper are fixed and without further mid-sequence code changes, the discipline Study 3 held after its own D1/D2 fixes. (ii) An empirical calibration of the enforced budget's power under realistic effect sizes, in the style of Study 2's A/A false-accept calibration. (iii) A longer horizon, to see whether arm A1 ever proposes a meta-scope edit at all, and whether `YieldGate`'s synthetic-world measurement (itself fixed this session) produces a non-zero signal in practice. (iv) The still-open judge-coverage precondition (`study4/README.md`): an explicit statement of which stated rules the judge does and does not test.

Limitations. One model, one provider (`openai/gpt-oss-120b` via Groq). One machine, one session, four process launches. Thirteen cycles is a small sample for any claim about the monitor's long-run behaviour. The pilot's own code changed mid-session, so it cannot be read as a controlled measurement of anything except the two things Table 2 isolates directly: the restart bug and its fix. `runs/study4_live` is not committed to the repository (it is covered by `.gitignore`, matching

how live-run folders are treated throughout this repository); the numbers in Tables 1–2 are this paper's durable record of it, reproduced from the ledger as shown in Appendix A.

6 Related work

e-LOND [12] and the broader game-theoretic, anytime-valid statistics framework [8] are what `kernel/stats.py`'s `ELond` implements; sequential testing with the same anytime-valid motivation goes back to Wald [10]. Repeated selection on one distribution inflating the measured gain of the winner [3] is the adjacent failure mode a held-out family and a two-stage gate are built to catch in this kernel; this paper's subject is the sibling failure mode of under-counting how many tests have already been run, not of counting them on the wrong data. Self-improving code agents that validate changes empirically [13], make the improvement procedure itself editable [14], or score a parent by its descendants' productivity [11] sit in the same space as the arm-A1 question Study 4 is building toward; formal self-reference goes back to Schmidhuber [9], and Chen et al. [1] surveys the field. Proxy-objective over-optimisation [2, 4] and a convinced self-play judge [15] are the reward-hacking risks this kernel's sealed verifier and coarse feedback are designed against; nothing in this paper's pilot exercised that threat model, since every rejected candidate here was a legitimate, correctly-implemented fix.

7 Conclusion

`kernel/orchestrator.py` resets its online false-discovery budget every time a new process starts; a long-lived guarded loop is made of many such processes. We fixed this without touching `kernel/`, by replaying the ledger's own history through an existing extension point in `kernel/pipeline.py`, and verified the fix both in a controlled test and live, where a level predicted from the ledger before a run matched the run exactly. A 13-cycle pilot against a live model then shows the bug and the fix in the same ledger, and shows an enforced, cold-started budget doing what it is specified to do: rejecting seven increasingly convincing candidates, including one at $p_{\text{better}} = 0.9999$, because none of them was the first discovery that would have earned the sequence more room. Nothing was accepted. We make no claim beyond that: this is a bug, a fix, and a transparent record of what thirteen real cycles under the fixed code actually did.

Reproducibility. Code: `https://github.com/abdelkabirouadoukou/si-loop`. Tests: `python -m pytest -q tests` (155 tests at the final commit of this session; the new ones for this paper's fix are listed in Appendix B). The fix: `study4/fdr_replay.py`. The pilot ledger analysed here is not committed (see Appendix A for its path and the exact commands used to produce every number in Section 4). The locked pre-registration for the runner is `results/study4/prereg.json`; `python run_study4.py -check` must print `pre-registration OK`.

Disclosure. Substantial AI assistance was used to write the code and to draft this paper, within the same session that found and fixed the bug it reports. The author directed the work, chose the design decisions (including enforcing the monitor for arm A1), and is responsible for the content. All numbers in Section 4 are reproduced directly from the ledger, not recalled from memory; the exact commands are in Appendix A.

References

[1] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026.

[2] Thomas Coste, Usman Anwar, Robert Kirk, and David Krueger. Reward model ensembles help mitigate overoptimization. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02743.

[3] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248): 636–638, 2015.

[4] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2210.10760.

[5] Abdelkabir Ouadoukou. A controlled ablation study of a guarded self-improvement loop with a competitive global workspace in a toy setting. Technical report, Independent researcher, 2026. Study 1 of this repository; not peer reviewed.

[6] Abdelkabir Ouadoukou. Guarded recursive self-improvement: Bounding creeping degradation and reward hacking in autonomous agent loops. Technical report, Independent researcher, 2026. Study 2 of this repository (scripted proposer); not peer reviewed.

[7] Abdelkabir Ouadoukou. Guarded self-improvement with a live language-model proposer: Replication, transfer, and two judge defects. Technical report, Independent researcher, 2026. Study 3 of this repository (live LLM proposer, solver-only arm); not peer reviewed.

[8] Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023.

[9] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. *Artificial General Intelligence (Cognitive Technologies), Springer*, 2007. arXiv:cs/0309048.

[10] Abraham Wald. Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2):117–186, 1945.

[11] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-Gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.21614.

[12] Ziyu Xu and Aaditya Ramdas. Online multiple testing with e-values. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238 of *PMLR*, 2024.

[13] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[14] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *arXiv preprint arXiv:2603.19461*, 2026.

[15] Chenyu Zhou. More convincing, not more correct: Self-play reward hacking of reference-free LLM judges. *arXiv preprint arXiv:2607.05904*, 2026.

A Reproducing every number in Section 4

```
python3 -c "
import json
from collections import Counter
with open('runs/study4_live/runs/ledger.jsonl') as f:
    recs = [json.loads(l) for l in f if l.strip()]
ends = [r for r in recs if r.get('ev') == 'end']
print('total end records:', len(ends))
print('verdicts:', Counter(r['verdict'] for r in ends))
print('error_types:', Counter(r['error_type'] for r in ends))
print('scopes:', Counter(r.get('scope') for r in ends))
fdr_tests = [r for r in ends if (r.get('detail') or {}).get('fdr')]
for i, r in enumerate(fdr_tests, 1):
    f = r['detail']['fdr']
    print(i, r.get('p_better'), r.get('delta'), f.get('level'), f.get('hit'))
"
python3 -c "
from study4.fdr_replay import replay_elond_state
e = replay_elond_state('runs/study4_live/runs/ledger.jsonl', 0.05)
print(e.t, e.d, e.level())
"
```

The second command reproduces the final state (t=7, d=0, next level ≈ 0.000475) reported at the end of Section 4; run before test 5 of the pilot, it is also the exact computation that produced the 0.0012158542037080532 prediction Table 2 reports as matched.

B Code map for this paper's contribution

- `study4/fdr_replay.py`: `replay_elond_state`, `attach_persistent_fdr_budget` (Section 3).
- `study4/run_live.py`: calls `attach_persistent_fdr_budget` once per process, before the first cycle, for both arms.
- `tests/test_study4_fdr_replay.py`: the controlled verification. It builds two independent `Orchestrator` instances sharing one ledger and checks that the second continues the first's exact (t, d).
- `study4/yield_gate.py`, `study4/live_meta.py`: the two other bugs found while building the arm-A1 runner (a mismatched synthetic verifier, and arm-A1 editing rights for `agent/meta/policy.py`). Out of this paper's scope, but documented in `study4/README.md`.
- `kernel/pipeline.py`: `TwoStageJudge.__init__`, `kernel/stats.py`: `ELond`: the unmodified kernel code this fix hooks into.

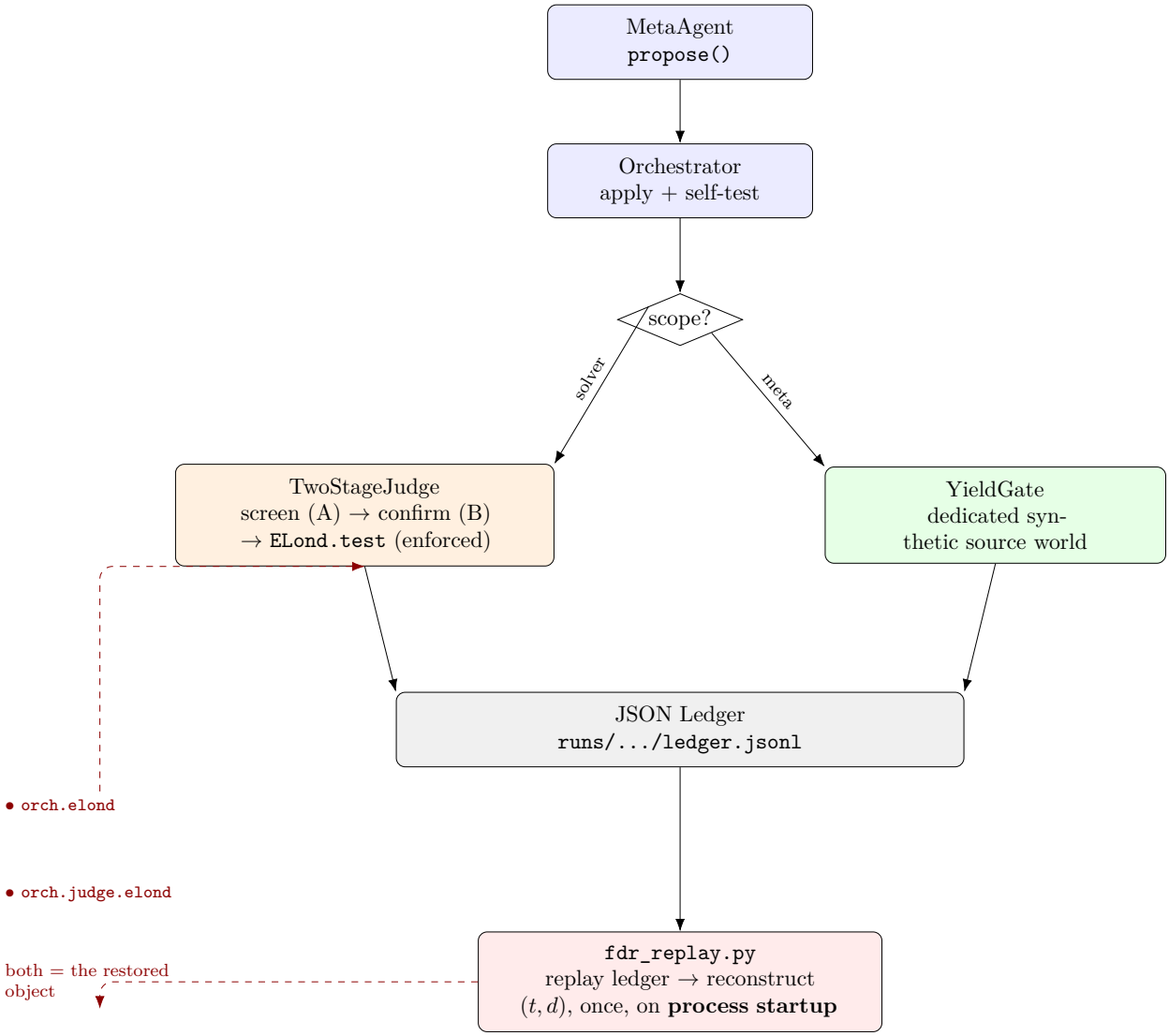


Figure 1: One cycle. A solver-scope patch goes through the two-stage judge and, on the correctness path, the enforced e-LOND test (folded into the same box: it is one call inside `TwoStageJudge.judge`, not a separate pipeline stage); a meta-scope patch (arm A1, `agent/meta/policy.py`) is instead judged by `YieldGate` against a dedicated synthetic world, never the live repository (a second bug fixed this session, `study4/yield_gate.py`). Every outcome is written to the ledger regardless of branch. The dashed path, executed once per process at startup, not per cycle, is this paper's fix: `fdr_replay.py` replays the ledger's history into a reconstructed (t, d) and installs it on both `orch.elond` and `orch.judge.elond`, the same object at construction time, before the next cycle can call `ELond.test`.

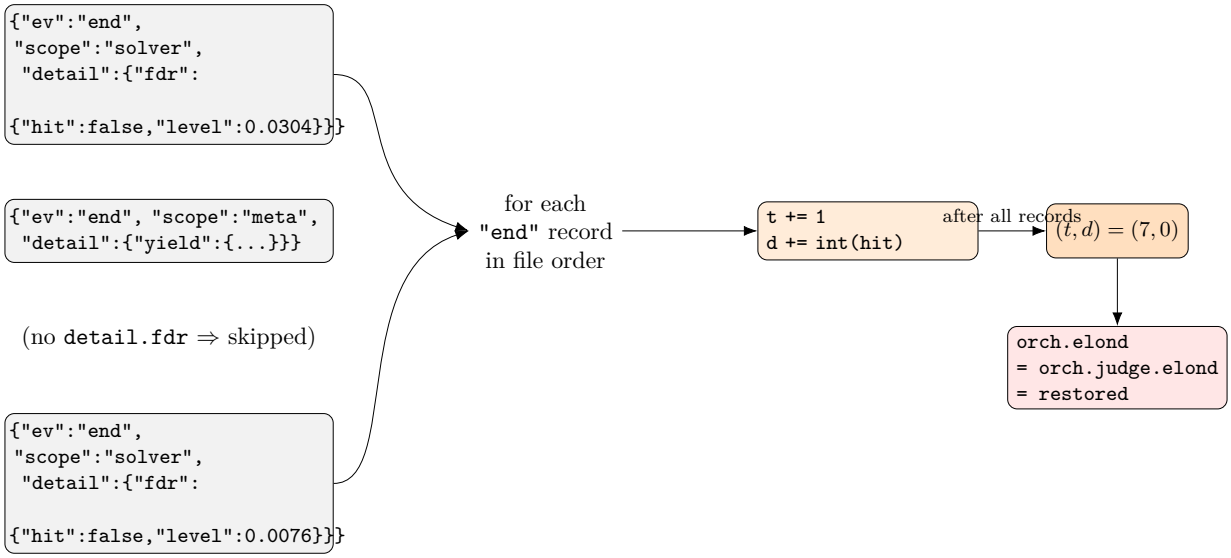


Figure 2: `replay_elond_state`: every ledger `"end"` record that carries a `detail.fdr` entry (a correctness-path test actually ran) contributes one (t+=1, d+=int(hit)) step, in file order; records with no such entry (a meta-scope yield test, or a cycle that never reached the confirm stage) are skipped. The reconstructed state is installed on *both* `orch.elond` and `orch.judge.elond`, which are the same object at construction time, so a partial update would silently desynchronise them.

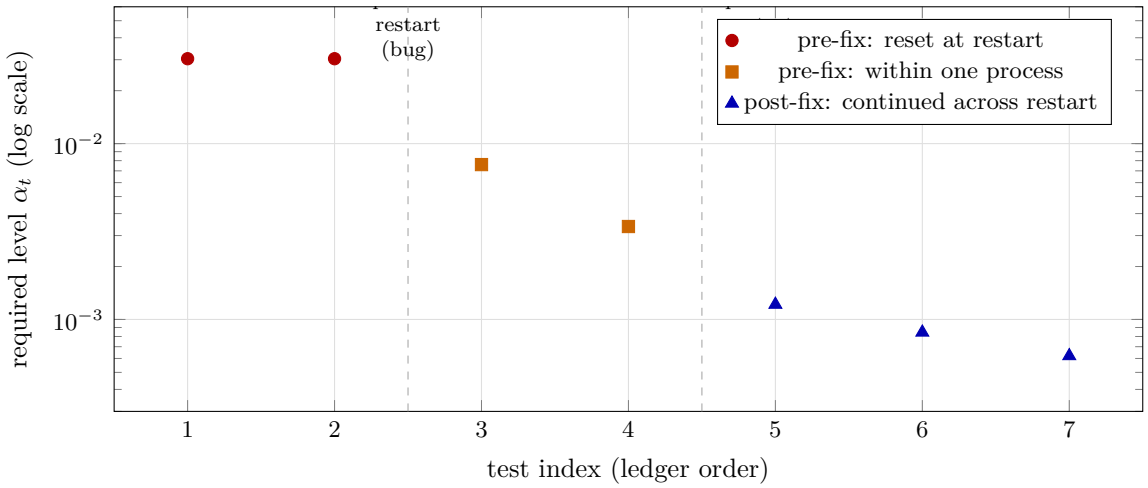


Figure 3: Required significance level α_t for each correctness-path test, in ledger order. The flat segment at tests 1–2 is the bug: a process restart handed back the generous first-test level instead of continuing the decline. From test 4 to test 5, a second process restart correctly continues the same declining sequence, because `fdr_replay.py` had been applied by then; test 5's level was computed from the ledger and published before that run, and matched.